



Plant Management System

Group C

Semi-Automatic Information and Knowledge Systems

**Ambrus Toth
Florian Krieger
Viktor Hoffmann**

Few Competency Question Examples

Our Plant Management System should be able to help both customers and the owner of the shop.



Customer perspective:

- Which plants grow in poor soil?
- Which low-light plants tolerate high soil salinity?
- What easy-care climbing plants cost under 30 eur?
- Which plants are edible and have edible leaves?

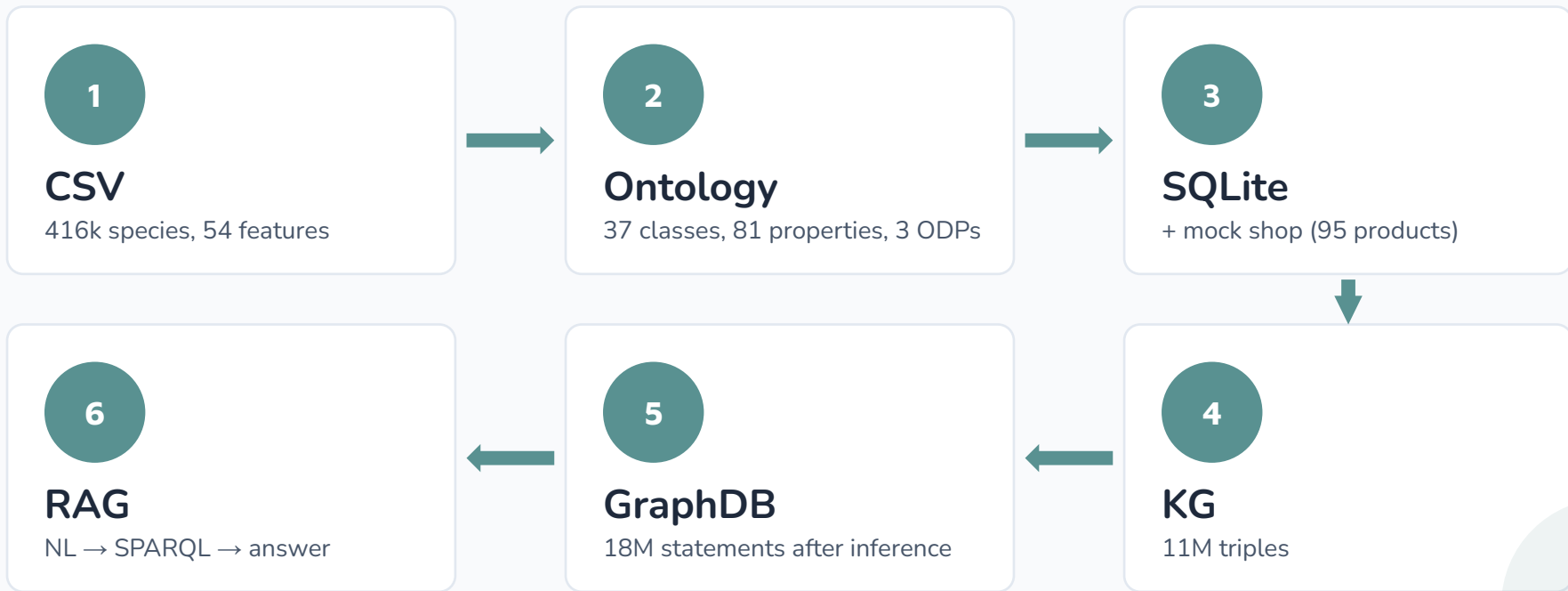
Shop Owner perspective:

- Which shop products require warm temperature, and have been on the shelf for a long time?
- What is our total stock quantity of all plants belonging to the Araceae family?
- Do we have enough Orchidaceae stock for an order of 5?

Pipeline



Six stages turn raw plant data into a queryable knowledge base.



Ontology Design



37

classes

48

object properties

33

datatype properties

Modelling highlights

- Plant parts modelled as separate classes (Flower, Fruit, Foliage, Root, Stem - subclasses of PlantPart), each with color and texture.
- A single plant species can map to multiple shop products.
- Heavy axiomatisation keeps taxonomy and products consistent.

OWL2 feature	Meaning
<code>owl:disjointUnionOf</code>	temperature categories
<code>owl:propertyChainAxiom</code>	taxonomy transitivity
<code>owl:FunctionalProperty</code>	one family per plant
<code>owl:hasKey</code>	product ID uniqueness
<code>owl:AllDisjointClasses</code>	seasons, plant parts

Design Patterns



Componency ODP:

Flower, Foliage, and Fruit have their own CSV attributes (color, texture, months). Modeling them as separate classes enables property scoping and the Componency ODP.

Class	Superclass	CSV
PlantPart	owl:Thing	— (abstract)
Flower	PlantPart	flower_color, flower_conspicuous
Foliage	PlantPart	foliage_color, foliage_texture
Fruit	PlantPart	fruit_color, fruit_conspicuous,fruit_months

AgentRole ODP:

Plant individual can have multiple roles, it can be both "Edible" and "Ornamental" simultaneously. We implement these roles as separate individuals, linking them to the plant with **hasPlantUse** property.

Class	Superclass	CSV
PlantUse	owl:Thing	— (abstract)
EdibleUse	PlantUse	edible = true
VegetableUse	PlantUse	vegetable = true
OrnamentalUse	PlantUse	decorative use (schema-level)

N-ary relation ODP:

PlantDistribution reification class presents a 3-way relation Plant × Region × Status which allows to retrieve the status of the species.

Class	Superclass	Notes
PlantDistribution	owl:Thing	Reification node: Plant × Region × Status
DistributionStatus	owl:Thing	NativeStatus, IntroducedStatus, EndemicStatus, AbsentStatus

OOPS & Knowledge Graph



R2RML results and OOPS! validator summary

OOPS! Validator Findings

Critical Logical Issues

- **Risky Transitivity (P29):** Verify hasPart/isPartOf transitive cascades to prevent incorrect classifications.
- **Domain/Range Intersections (P19):** 16 properties flagged for multiple ranges; check for unintended intersections.

Important Metadata

- **Missing License (P41):** Add a standard open license declaration to the ontology header.

Minor Housekeeping

- **Missing Inverses (P13):** Add useful inverse declarations (19 missing).
- **Unconnected Elements (P04):** Link or delete isolated classes (Season, TaxonomicRank, TimeInterval).

Knowledge Graph Statistics

Content Inventory

Plants **416,473**

Families **580**

Genera **15,540**

Shop products **95**

Total RDF Triples

~11.1M

SHACL Validation



SHACL validates identity, links, value ranges, and shop data in the generated KG.

✓ 1. Identity

Plant has exactly one ID and scientific name

✓ 2. Taxonomy

Genus and family links use the correct classes

✓ 3. Numeric ranges

pH, light, humidity, stock, and price stay valid

✓ 4. Plant structure

Months, seasons, parts, colors, and distributions are linked correctly

✓ 5. Shop data

Products have price, stock, care level, temperature, and plant link

Validation result: conforms

Alignment with Plant Ontology



Linking our local plant management ontology to a shared botanical vocabulary.

Why alignment?	Local concept	Plant Ontology concept	Relation
• gives our plant-part classes a standard botanical meaning • improves interoperability with external plant datasets • makes future extension easier • keeps shop-specific concepts local	Flower	PO flower	exact match
	Fruit	PO fruit	exact match
	Root	PO root	exact match
	PlantPart	PO plant structure	close match
	Foliage	PO leaf	close match

Manual and conservative alignment: exact matches only where the meaning is clearly the same.

RAG Architecture



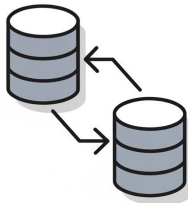
User query:



Natural language input is processed (e.g., "Find plants with blue flowers").



Schema link:



Retrieves relevant ontology terms (Classes/Properties) and entity candidates from GraphDB.



SPARQL generation:

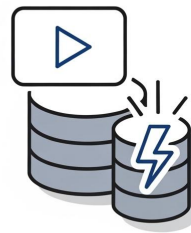


The LLM model translates the natural language into a SPARQL SELECT query.

We are using OpenAI's gpt-4o-mini model.



Execution:



GraphDB executes the query, returning the facts from the graph.

Execution is possible via CLI or through a dedicated web UI



Settings

Model

gpt-4o-mini

GraphDB endpoint

http://localhost:7200/repositories/plar

☒ Show generated SPARQL

Deploy ⓘ

Plant Management KG

Natural language query interface -- KG-based RAG with SPARQL generation

Ask a question about the plant knowledge graph...



Example questions:

What is the total stock of all Araceae plants in the shop?

Do we have enough Orchidaceae stock for an order of 5?

What easy-care climbing plants cost under EUR 30?

Which shop products require warm temperature and have been on the shelf for a long time?

Which low-light plants tolerate high soil salinity?

Which plants are edible and have edible leaves?

Which plants have blue flowers?

Which plants grow in poor soil?

Which plants bloom in January?

Which plants belong to family Pinaceae?

What is the maximum height of Pinaceae plants?

Answer

The maximum height of Pinaceae plants is 9900 cm for the species *Pseudotsuga menziesii*.

Generated SPARQL

```
SELECT ?scientificName ?maximumHeightCm WHERE { ?plant a plant:Plant ; plant:hasScientificName ?scientificName ; plant:hasMaximumHeightCm ?maximumHeightCm ; plant:belongsToFamily ?family . ?family rdfs:label "P
```

Lessons learned, challenges:



Ontop failure:

- The standard Ontop 5.4.0 did not work for materialization
- It failed to generate SQL for SQLite when multiple triples maps produce the same predicate
- We had to write a custom Python materializer that follows the same R2RML structure

Data sparsity:

- Trefle dataset is large (416k species) but many fields are empty (not all plants have color, edible parts etc.)
- Certain CQs return very few/zero results (e.g., “plants that have edible leaves and roots”)

LLM SPARQL generation:

- LLM invented property names that do not exist in our ontology
- Mixed up boolean representation (used “true” instead of 1 for isEdible)
- Mixed up aggregate (SUM) with non-aggregated variables (?scientificName)